

TaskFlow — Kanban JavaScript

Complete JavaScript source code

```
let defaulttasks = [
  {
    id: "task-101",
    title: "Design System Accessibility Review",
    description: "Audit color contrast ratios and keyboard navigation across all core UI components.",
    status: "backlog",
    tag: "UI/UX",
    tagClass: "tag-ui",
    priority: "Low",
    priorityClass: "priority-low",
    subtasks: "1/3"
  },
  {
    id: "task-102",
    title: "User Auth & JWT Flow",
    description: "Set up login/signup routes, password hashing, and token validation in Express.",
    status: "todo",
    tag: "Backend",
    tagClass: "tag-backend",
    priority: "High",
    priorityClass: "priority-high",
    subtasks: "0/4"
  },
  {
    id: "task-103",
    title: "Refactor Filter Logic",
    description: "Optimize category filtering using native array method chains to improve DOM render speed.",
    status: "in-progress",
    tag: "Frontend",
    tagClass: "tag-frontend",
    priority: "Medium",
    priorityClass: "priority-medium",
    subtasks: "2/2"
  },
  {
    id: "task-104",
    title: "Database Schema Migration",
    description: "Update task document structure to support custom tag arrays and assignee IDs.",
    status: "review",
    tag: "Backend",
    tagClass: "tag-backend",
    priority: "Medium",
    priorityClass: "priority-medium",
    subtasks: "3/3"
  },
  {
    id: "task-105",
    title: "Project Setup & Base Styles",
    description: "Configure HTML structure, CSS custom variables, and Flexbox layout for sidebar.",
    status: "done",
    tag: "Frontend",
    tagClass: "tag-frontend",
    priority: "Low",
    priorityClass: "priority-low",
  }
]
```

```

        subtasks: "2/2"
    }
];

function saveTasks(array) {
    localStorage.setItem("tasks", JSON.stringify(array));
}

function loadTasks() {
    const storedTasks = localStorage.getItem("tasks");
    if (storedTasks) {
        return JSON.parse(storedTasks);
    }

    return defaulttasks;
}

let tasks = loadTasks();

function renderTasks(array) {
    const columns = document.querySelectorAll(".kanbancolumn");

    columns.forEach(column => {
        let status = column.dataset.status;
        const temp = array.filter(task => task.status === status);

        let content = column.querySelector(".taskList");
        column.querySelector(".taskCount").innerHTML = temp.length;

        const Allcardshtml = temp.map(task => `
            <div class="task-card" data-id="${task.id}" draggable = "true">
                <div class="card-header">
                    <span class="card-category ${task.tagClass}">${task.tag}</span>
                    <span class="card-priority ${task.priorityClass}">${task.priority}</span>
                    <button class="delete">Delete</button>
                </div>

                <h3 class="card-title">${task.title}</h3>
                <p class="card-description">${task.description}</p><br>

                <div class="card-footer">
                    <span class="task-subtasks">■ ${task.subtasks}</span>
                    <span class="task-id"># ${task.id}</span>
                </div>
            </div>
        `).join('');

        content.innerHTML = Allcardshtml;
    });
}

document.addEventListener("dragstart", (e) => {
    const card = e.target.closest(".task-card");
    if (!card) return;

    e.dataTransfer.setData("text/plain", card.dataset.id);

    setTimeout(() => card.classList.add('dragging'), 0);
});
document.addEventListener("dragend", (e) => {

```

```

    const card = e.target.closest(".task-card");
    if (!card) return;

    card.classList.remove('dragging');
  });

document.addEventListener("dragover", (e) => {
  if (e.target.closest(".kanbancolumn")) {
    e.preventDefault();
  }
});

document.addEventListener("drop", (e) => {
  e.preventDefault();

  const column = e.target.closest(".kanbancolumn");
  if (!column) return;

  const id = e.dataTransfer.getData("text/plain");
  const task = tasks.find(t => t.id === id);

  if (task) {
    task.status = column.dataset.status;
  }
  saveTasks(tasks);
  renderTasks(tasks);
});

saveTasks(tasks);
renderTasks(tasks);

const form = document.querySelector(".add");
const addTaskbtn = document.querySelector(".addTask");
const cancelBtns = document.querySelectorAll(".cancel");
const modalOverlay = document.getElementById("modalOverlay");

function openModal() {
  modalOverlay.classList.add("appear");
}

function closeModal() {
  modalOverlay.classList.remove("appear");
}

addTaskbtn.addEventListener("click", openModal);
cancelBtns.forEach(btn => btn.addEventListener("click", closeModal));

function createTask() {
  form.addEventListener("submit", (e) => {
    e.preventDefault();

    const formData = new FormData(e.target);

    const title = formData.get('title');
    const desc = formData.get('description');
    const status = formData.get('status');
    const priority = formData.get('priority');
    const tag = formData.get('tag');

    let tagClass = "none";

```

```

    if (tag === "UI/UX") {
      tagClass = "tag-ui";
    } else {
      tagClass = `tag-${tag.toLowerCase()}`;
    }

    const priorityClass = `priority-${priority.toLowerCase()}`;
    const id = `task-${Date.now()}`;

    const formattedPriority = priority.charAt(0).toUpperCase() + priority.slice(1);

    const newTask = {
      id: id,
      title: title,
      description: desc,
      status: status,
      tag: tag,
      tagClass: tagClass,
      priority: formattedPriority,
      priorityClass: priorityClass,
      subtasks: "0/5"
    };

    tasks.push(newTask);
    saveTasks(tasks);
    renderTasks(tasks);

    e.target.reset();

    closeModal();
  });
}

createTask();

document.addEventListener("click", (e) => {
  const delbtn = e.target.closest(".delete");

  if (!delbtn) return;

  const del = e.target.closest(".task-card");
  const taskId = del.dataset.id;
  const index = tasks.findIndex(t => t.id === taskId);
  tasks.splice(index, 1);
  saveTasks(tasks);
  renderTasks(tasks);
});

const filterbtn = document.querySelector("#filter");

filterbtn.addEventListener("click", () => {
  const element = document.querySelector(".filtermenu");
  element.classList.add("appearfilter");
});

const applyfilter = document.querySelector(".filtermenu");

applyfilter.addEventListener("submit", (e) => {
  e.preventDefault();
  const tagtofilter = document.querySelector('input[name = "ftag"]:checked');

```

```
const prioritytofilter = document.querySelector('input[name = "fpriority"]:checked');

const ftag = tagtofilter.value;
const fpriority = prioritytofilter.value;

const filteredtasks = tasks.filter(t => {
  const matchesTag = (ftag === "All") || (t.tag === ftag);
  const matchesPriority = (fpriority === "All") || (t.priority === fpriority);

  return matchesTag && matchesPriority;
});
saveTasks(filteredtasks);
renderTasks(filteredtasks);
applyfilter.classList.remove("appearfilter");
});

const closefilterbtn = document.querySelector(".closefilter");

closefilterbtn.addEventListener("click", () => {
  const element = document.querySelector(".filtermenu");
  element.classList.remove("appearfilter");
});

const searchBar = document.querySelector(".search");

searchBar.addEventListener("input", (e) => {
  const query = e.target.value.toLowerCase();

  const searchTasks = tasks.filter(t => {
    const tagmatch = t.tag.toLowerCase().includes(query);
    const prioritymatch = t.priority.toLowerCase().includes(query);
    const titlematch = t.title.toLowerCase().includes(query);
    const descriptionmatch = t.description.toLowerCase().includes(query);
    return tagmatch || prioritymatch || titlematch || descriptionmatch;
  });
  saveTasks(searchTasks);
  renderTasks(searchTasks);
});
```